KLAUSUR ZUR VORLESUNG ***"SOFTWARE ENGINEERING"*** **WINTERSEMESTER 2008/2009**	FH AC Hochschule Aachen
Prof. Dr.-Ing. A. Terstegge, FB9 Medizintechnik und Technomathematik	

Name:	Vorname:	Matrikelnummer:	Note:

Prüfungstag: 20.01.2008
Prüfungsdauer: 120 Minuten
Prüfungsumfang: 6 Aufgaben (11 Seiten)
Hilfsmittel: Vorlesungsfolien, Buch Brügge/Dutoit, UML-Übersicht, **KEINE** elektronischen Hilfsmittel oder Kommunikationsgeräte.

Hinweise:

1.) Verlangt sind alle Aufgaben.
2.) Bitte verwenden Sie keinen Rotstift / Bleistift.
3.) Bitte tragen Sie in den Kasten Ihren Namen und Ihre Matrikelnummer ein.
4.) Verwenden Sie für jede Aufgabe ein separates Lösungsblatt.
5.) Verweise auf Skriptseiten oder Übungsaufgaben werden nicht gewertet.
6.) Geben Sie dieses Deckblatt zusammen mit Ihren Lösungsblättern ab.
7.) Schreiben Sie auf alle Lösungsblätter unbedingt Ihren Namen und Ihre Matrikelnummer.

Nicht vom Studierenden auszufüllen

1	2	3	4	5	6				

Summe	Prozent

Aufgabe 1: Spezifikation mit UML

a) Gegeben seien folgende Anforderungen an ein Textverarbeitungssystem: Das Textverarbeitungssystem erlaubt es Peter Müller (und anderen Benutzern) Dokumente anzulegen und zu editieren. Ein Dokument kann Text und Bitmap-Grafik enthalten. Text besteht aus Abschnitten, jeder Abschnitt aus Zeichen. Ein Dokument enthält außerdem verschiedene administrative Informationen wie seinen Titel, seinen Autor, den Dateinamen, in dem es abgelegt ist, sowie das Datum der letzten Änderung.

Aufgaben:

- Welche Objekte/Klassen lassen sich im obigen Problemfall identifizieren?

- Welche Beziehungen (Assoziation, Aggregation oder Komposition) bestehen zwischen den Klassen? Zeichnen Sie das entsprechende Klassendiagramm in der UML-Notation.Berücksichtigen Sie Assoziationsnamen, Assoziationsenden (Rollen), Navigierbarkeit, Multiplizitäten und für die Klasse *Dokument* alle Attribute.

Aufgabe 2: Modellierung

a) Gegeben ist der folgende Java-Code:

```java
public class Examination {
    int att1;
    public int att2;
    public static double pi=3.1415;
    private boolean att3;
    protected short att4;
    final String att5 = "Test";
    KlasseB att6;
    java.util.Vector<String> att7;
    Object getAtt8 () {
        // Berechne Wert für att8
        return wert;
    }

    public static void op1(){
        // Implementierung
    }
    private int op2(final int param1) {
        // Implementierung
        return wert;
    }
    protected void op3(KlasseC param3) {
        // Implementierung
    }
    KlasseB op4(String param3[]) {
        // Implementierung
        return wert;
    }
}
```

Geben Sie das vollständige Klassendiagramm (mit Attributen!) an, das diesem Code zu Grunde liegt!

b) Ein Bleistift besteht bekanntermaßen aus einer Mine, einem Holzschaft und einer Schutzlackierung.

Entsprechend besteht ein Kugelschreiber aus einer Mine, einer Hülle und ein wenig Mechanik.

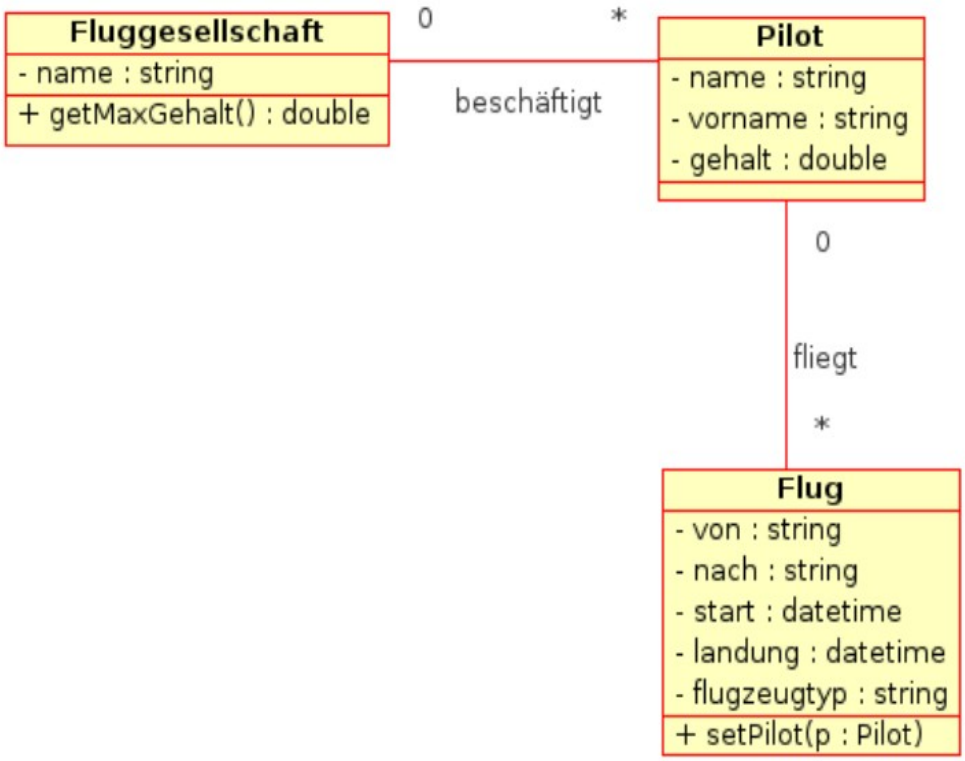
Geben Sie ein sinnvolles Klassendiagramm mit Attributen für Bleistift und Kugelschreiber an, in dem insbesondere die Zusammensetzung der beiden Teile deutlich erkennbar ist!

Aufgabe 3: Object Constraint Language

a) Geben Sie ein Beispiel für eine Integritätsbedingung, die nicht in UML modelliert werden kann.

b) Nennen Sie zwei Arten von OCL Contraints. Beschreiben Sie diese kurz.

c) Gegeben sei folgendes Klassendiagramm:



Formulieren Sie OCL-Aussagen um folgende Sachverhalte zu modellieren:

- Keine Fluggesellschaft beschäftigt einen Piloten, der mehr als die Gehaltsobergrenze (Operation `getMaxGehalt()`) im Monat verdient.

- Stellen Sie sicher, dass die Fluggesellschaft `Singapore Airlines` die einzige ist, die auch Flüge mit Flugzeugen vom Typ `A380` anbietet.

- Spezifizieren Sie Vor- und Nachbedingungen für die Methode `setPilot(p : Pilot)`in OCL, die sicherstellen, dass ein Pilot niemals zwei Flüge gleichzeitig durchführen kann. Außerdem muss er nach einem Flug mindestens eine Pause von 2 Stunden haben, bis er wieder starten kann.

Aufgabe 4: Testverfahren

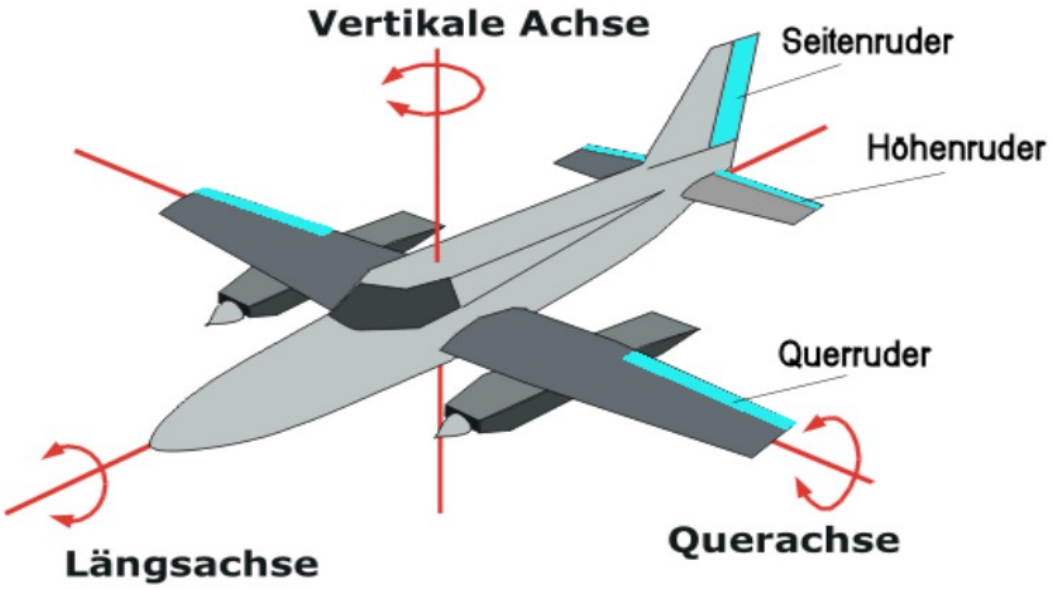
Bei kontrollflussbezogenen Kontrollverfahren versucht man ein gezieltes Durchlaufen (von Teilen) der Kontrollstrukturen, durch geeignete Gestaltung der Tesfälle, zu erreichen. Es sei folgender Programmteil gegeben:

```
IF x>12 OR y<7
  THEN y:=f(2,x)
  ELSE y:=g(x, 3)
  z=y*2
  ...
```

a) Geben Sie Testdaten an, mit deren Hilfe Sie eine Zweigüberdeckung für diesen Programmteil sicherstellen. Erfüllen Ihre Testdaten auch das Kriterium für die Mehrfach-Bedingungsüberdeckung? Falls nicht, dann erweitern Sie Ihre Testdaten.

Aufgabe 5: Modellierung der Zustände und Test einer Steuersoftware für Flugzeughöhenruder

Der Luft- und Raumfahrtzulieferer SDAE entwickelt eine Steuersoftware für neues Steuergerät eines Kleinflugzeugs. Es ist für die Ansteuerung des Höhenruders am Flugzeug zuständig. Das Flugzeughöhenruder dient zur Drehung eines Flugzeuges um seine Querachse. Beim Flugzeug verändert sich dadurch der Anstellwinkel der Tragfläche und damit der Auftrieb.



Das Steuergerät unterstützt folgende Funktionen:

- Der Pilot steuert die Bewegung des Höhenruders durch eine Auf- oder Abwärtsbewegung des Steuerknüppels.
- Bei Betätigung des Steuerknüppels in die jeweilige Richtung wird eine Bewegung des Höhenruders in die entsprechende Richtung gestartet.
- Die Auf- oder Abwärtsbewegung des Höhenruders wird automatisch gestoppt, sobald das Höhenruder vollständig in die jeweilige Richtung ausgelenkt ist.
- Wird während einer Auf- oder Abwärtsbewegung des Höhenruders der Steuerknüppel in die entgegengesetzte Richtung bewegt, ändert sich die Auf- oder Abwärtsbewegung des Höhenruders entsprechend.
- Wenn ein mechanischer Fehler des Höhenruders erkannt wird, beispielsweise durch Vereisung, wird das Höhenruder aus Sicherheitsgründen in seine horizontale Ausgangsposition bewegt.

Das Steuergerät selbst ist ein Mikrocontrollerboard. Es ist über ein spezielles Bussystem mit den Sensoren und Aktoren des Höhenruders vernetzt. Das Einlesen der Sensormesswerte sowie die Ausgabe von Kommandos an die Aktoren erfolgt durch Senden bzw. Empfangen entsprechender Nachrichten über den Bus.

Im Einzelnen setzt sich das Flugzeughöhenruder (engl. *Elevator*) aus folgenden Sensoren und Aktoren zusammen:

- Ein Steuerknüppel mit entsprechender Sensorik zur Erfassung der gewünschten Ansteuerung des Flugzeughöhenruders. Bei Betätigung der Sensorik werden die Nachrichten `ElUp` bzw. `ElDown` ausgegeben.
- Einem Stellmotor zum Bewegen des Flugzeughöhenruders. Dieser kann mit Hilfe der Kommandos `MoveElUp`, `MoveElDown` und `ElStop` angesprochen werden. Durch ständiges Messen des Drehmoments ist er in der Lage zu erkennen, ob die Bewegung des Flugzeughöhenruders fehlerfrei funktioniert. Im Fehlerfall sendet der Motor die Nachricht `ElFailureDetected` aus.
- Zwei Positionssensoren, die melden wenn das Flugzeughöhenruder in eine der beiden maximalen Stellpositionen ausgelenkt ist. Ist dies der Fall, so wird dies durch die Nachrichten `MaxUp` bzw. `MaxDown` signalisiert.

<u>Frage 5.1: Zustandsautomat</u>
Zeichnen Sie ein Zustandsdiagramm in UML-Notation für das oben spezifizierte Steuergerät. Ermitteln Sie hierzu die notwendigen Zustände, Ereignisse und Ergebnisse. Desweiteren geben Sie die erforderlichen Übergänge im Zustandsdiagramm an.
Hinweis: Nach Eintreten des Fehlerfalls kann ein Reset-Befehl ausgegeben werden, womit das System in einen sicheren Zustand überführt wird.

<u>Frage 5.2: Pseudocode</u>
Geben Sie eine Implementierung für den von Ihnen entworfenen Zustandsautomaten aus Frage 5.1 in Form von Pseudocode an. Erstellen Sie ein funktionsfähiges Grundgerüst, das lediglich die Einstiegspunkte zu allen Zustände in Ihrem Zustandsautomaten aufweist. Bestimmen Sie einen Zustand Ihrer Wahl und geben Sie für diesen eine detaillierte Implementierung mit allen Übergängen und Aktionen an.

<u>Frage 5.3: Testtreiber</u>
Um die Steuersoftware des Flugzeughöhenruders risikolos testen zu können, wird anstelle der realen Sensoren und Aktoren zunächst ein PC-basierter Emulator verwendet. Dieser ist über das Bussystem mit dem Steuergerät verbunden und kann sämtliche Nachrichten der normalerweise vorhandenen Sensorik und Aktorik generieren bzw. abfangen.
Benutzen Sie die Funktionen `SendMsg()` und `WaitForMsg()` um mit den Sensoren und Aktoren zu kommunizieren. Mit SendMsg() kann eine Nachricht, die als Parameter zu übergeben ist, über das Bussystem gesendet werden. Beim Aufruf von `WaitForMsg()`, der ohne Übergabe von Parametern erfolgt, wird die nächste eintreffende Nachricht abgewartet und als Rückgabewert übergeben.
Entwickeln Sie einen Testtreiber für den PC-basierten Emulator in Pseudocode um folgenden Ablauf zu überprüfen: Der Steuerknüppel wird nach oben betätigt – das Flugzeughöhenruder bewegt sich nach oben – das Flugzeughöhenruder ist vollständig ausgelenkt – der Stellvorgang wird gestoppt.

Benutzen Sie zusätzlich die Funktion `TrackMsg()` um sämtliche ausgesendeten und empfangenen Nachrichten am Bildschirm anzuzeigen. Die auszugebende Nachricht kann dabei als Parameter an TrackMsg() übergeben werden. Der Test ist erfolgreich, wenn vom Steuergerät die jeweils richtigen Kommandos an den Stellmotor abgesetzt werden. In diesem Fall liefert der Testtreiber TRUE zurück.

Sobald ein falsches Kommando erkannt wird, wird der Test abgebrochen und FALSE zurückgeliefert.

Frage 5.4: Testfallermittlung

a) Ein C1-Abdeckungstest besagt, dass die Wahl des Testfalls so zu erfolgen hat, dass alle Zweige eines Programms mindestens einmal durchlaufen werden. Wie lautet das entsprechende Kriterium, wenn man die Idee von C1-Abdeckungstests auf Zustandsautomaten überträgt? Auf welche Weise erhält man dann die zugehörigen Testfälle?

b) Geben Sie einen möglichen Testfall zur C1-Abdeckung des von Ihnen entworfenen Zustandsautomaten aus Frage 5.1 an.

Aufgabe 6: Verständnisfragen

Frage 5.1: Systementwurf
Erklären Sie den Begriff Schichtenarchitektur und nennen Sie ein Beispiel. Welche anderen Architekturen beim Systementwurf kennen Sie? Nennen Sie jeweils ein Beispiel!

Frage 5.2: Qualitätssicherung
Was ist der Unterschied zwischen konstruktiver Qualitätssicherung und analytischer Qualitätssicherung? Welche von beiden ist wichtiger? Begründen Sie Ihre Antwort.

Frage 5.3: Test
Erklären Sie den Unterschied zwischen einem Black-Box- und einem White-Box-Test. Welche Gründe sprechen für den Einsatz eines White-Box-Tests, welche für den Einsatz eines Black-Box- Tests? Nennen Sie jeweils 2 Gründe.

Frage 5.4: Softwarequalität
Nennen Sie 5 Qualitätsmerkmale für Software. Können alle Qualitätsmerkmale gleichermaßen erreicht werden? Begründen Sie Ihre Aussage!

Frage 5.5: Programmierstil

Finden Sie drei negative Eigenschaften des Programmierstils in dem folgenden Java-Quellcode.

```
public class QuickSort
{
     private int[] a;
     private int n;

     public void doSomething(int[] a)
     {
          this.a=a;
          n=a.length;
          doThing(0, n-1);
     }

     private void machWas(int /* low value */ r, int /* hoher Wert */ s)
     {
          int i=r, j=s;
          int x=a[(r+s)/2];
          while (i<=j)
          {
               while (a[i]<x) i++;
               while (a[j]>x) j--;
               if (i<=j)
               {
                 ex(i, j);
                 i++; j--;
               }
             }
             if (r<j) doThing (r, j);
             if (i<s) doThing (i, s);
     }

   private void ex(int i, int j)
   {
     int t=a[i];
     a[i]=a[j];
     a[j]=t;
   }
}
```